Informe de Pruebas Unitarias del Backend — EduAirControl

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Tecnólogo en Análisis y Desarrollo de Software – 3145556, Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software – 3145556

Todos los instructores del programa

11 de octubre de 2026 · 2025-2026

Resumen

Este informe reporta el estado de las **pruebas unitarias del backend** de EduAirControl: siete servicios Java 17 / Spring Boot 4 verificados el **11 de octubre de 2026** con `./mvnw -B clean verify` sobre sus ramas `develop`. El resultado de la corrida es de **294 pruebas, 0 errores y 0 fallos** en 44 clases de prueba (≈167 s de tiempo de prueba acumulado).

De ese total, **100 pruebas en 15 clases son pruebas unitarias puras** (JUnit 5 + Mockito + AssertJ, sin contexto Spring ni base de datos): son el foco de este documento. Las restantes se clasifican en pruebas de rebanada (`@SpringBootTest` con MockMvc sobre H2 en memoria), pruebas de integración con PostgreSQL real mediante Testcontainers y pruebas de filtros/ validación del API Gateway. El script `infra/scripts/smoke.sh` **no es una prueba unitaria**: es una prueba de humo extremo a extremo (E2E) que exige el stack completo levantado y que se describe en la Sección 6 para distinguirla con claridad.

El informe concluye con seis hallazgos, entre ellos la ausencia de medición de cobertura (no hay JaCoCo configurado, pese a que RNF6 pide ≥ 80 % de líneas), la omisión de `ms-user-experience` en la matriz de integración continua y el riesgo de falsos positivos por artefactos compilados obsoletos cuando el pipeline no ejecuta `clean`.

Palabras clave: pruebas unitarias, JUnit 5, Mockito, Testcontainers, H2, integración continua, cobertura de código.

Índice

Resumen 2

1. Introducción 4

2. Definiciones: qué es y qué no es una prueba unitaria aquí 4

3. Metodología de la corrida 6

4. Resultados por servicio 7

5. Inventario de las suites unitarias y trazabilidad con los requisitos 8

6. Lo que no es unitario (y por qué importa distinguirlo) 10

    6.1 Pruebas de rebanada sobre H2 (89 pruebas) 10

    6.2 Pruebas con Testcontainers (65 pruebas) 11

    6.3 El smoke E2E: infra/scripts/smoke.sh 11

7. Integración continua 11

8. Hallazgos y recomendaciones 12

9. Conclusiones 13

Referencias 15

Anexo A. Comandos de la corrida 16

Anexo B. Detalle por clase 16

Informe de Pruebas Unitarias del Backend — EduAirControl

1. Introducción

El presente informe documenta la situación real de las pruebas automatizadas del backend de EduAirControl, con énfasis en las **pruebas unitarias**. Su propósito es triple: (a) responder de forma verificable qué pruebas existen, cuántas son, qué cubren y en qué estado están; (b) dejar un punto de referencia reproducible para las fases de diseño e implementación; y (c) alimentar la estrategia de calidad del proyecto (`docs-ea-control/11-quality/`).

La información proviene de una corrida fresca de las suites de los siete servicios del repositorio de trabajo, ejecutada el 11 de octubre de 2026, y del análisis estático de cada clase de prueba (anotaciones, dependencias y método bajo prueba). Los identificadores `RFx.y`, `RNFx` y `CU-xx` son los de la **Especificación de Requisitos de Software (Entregable 02)**, los mismos que usa el `03_Analisis_Software`, de modo que ambas piezas comparten una única nomenclatura.

El documento se organiza como sigue: Sección 2 (definiciones: qué es y qué no es una prueba unitaria en este proyecto), Sección 3 (metodología de la corrida), Sección 4 (resultados por servicio), Sección 5 (inventario de las suites unitarias y su trazabilidad con los requisitos), Sección 6 (lo que no es unitario: rebanadas, Testcontainers y el smoke E2E), Sección 7 (integración continua), Sección 8 (hallazgos y recomendaciones), Sección 9 (conclusiones) y anexos con los comandos y el detalle por clase.

2. Definiciones: qué es y qué no es una prueba unitaria aquí

Para evitar la confusión que motiva este informe —«¿las pruebas unitarias son los test y el smoke.sh?»—, la tabla siguiente fija la taxonomía usada en todo el documento.

Tabla 1

Taxonomía de las pruebas automatizadas del backend

Tipo	Qué verifica	Aislamiento	Herramienta en este proyecto	Dónde corre
Unitaria pura	Lógica de dominio y de aplicación en aislamiento total: agregados, servicios de aplicación, validadores, constructores de eventos	Completo: sin contexto Spring, sin base de datos, sin red; dependencias simuladas con Mockito o constructores manuales	JUnit 5 + Mockito + AssertJ	`./mvnw verify`
De rebanada (slice)	Controladores y su configuración de seguridad con `MockMvc`, sobre una base H2 en memoria	Medio: contexto Spring mínimo, sin PostgreSQL real	`@SpringBootTest` + MockMvc + H2	`./mvnw verify`
De integración con contenedor	Repositorios, listeners de RabbitMQ y controladores contra **PostgreSQL real** (y RabbitMQ real en monitoreo)	Bajo: servicios reales en Docker	Testcontainers	`./mvnw verify` (requiere Docker)
E2E / humo	Flujos completos a través del API Gateway con el stack completo levantado	Ninguno: entorno real desplegado	`infra/scripts/smoke.sh` (bash + curl + jq)	Manual, sobre `docker compose up -d`

Con esa definición: **las pruebas unitarias de este proyecto son las 100 pruebas de la primera fila**, no las 294 totales, y el `smoke.sh` pertenece a la última fila: es una verificación de humo extremo a extremo, no una prueba unitaria, y además **no corre automáticamente en ninguna tubería de CI**.

3. Metodología de la corrida

Tabla 2

Condiciones de la corrida del 11 de octubre de 2026

Elemento	Valor
Fecha y hora de la corrida	11 de octubre de 2026 (UTC-5)
Comando ejecutado por servicio	`./mvnw -B clean verify`
Rama y commit verificados	`develop` (último `git pull` de cada repositorio; todos al día)
JDK	OpenJDK 21 (compilación con `--release 17`)
Motor de pruebas	Surefire (Maven) dentro de `verify`
Base de datos en pruebas	H2 en memoria (5 servicios) y PostgreSQL en contenedor (2 servicios)
Contenedores de prueba	Docker Engine 29.7.2 (Testcontainers levanta PostgreSQL 15 y, en monitoreo, RabbitMQ)
Criterio de éxito	`BUILD SUCCESS` con 0 errores y 0 fallos en cada servicio

Se usó `clean` de forma deliberada: sin él, Maven reutiliza clases compiladas de ejecuciones previas y hoy se comprobó que tres servicios reportaron **64 errores espurios** por artefactos de paquetes renombrados; tras `clean verify` los tres pasan. Este hecho está registrado como hallazgo H2 (Sección 8).

Nota sobre el API Gateway: su copia de trabajo local contiene una modificación sin commitear en `RedisStateStoreImpl.java` que rompe la compilación (ver hallazgo H5). Para no alterarla, la suite se verificó contra el commit `13232d4` de `develop` en un árbol de trabajo aislado, obteniendo 27/27 en verde.

4. Resultados por servicio

Tabla 3

Resultados de la corrida del 11 de octubre de 2026 (pruebas / errores / fallos / segundos)

Servicio	Puerto	Pruebas	Errores	Fallos	Seg.	Reparto por tipo
api-gateway	8080	27	0	0	12,9	6 unitarias; 6 de rebanada; 15 de filtros/validación
ms-security	8081	57	0	0	33,6	12 unitarias; 29 de rebanada; 16 de servicio con simulacros
ms-classroom-management	3002	33	0	0	15,7	10 unitarias; 23 de rebanada
ms-sensor-management	3004	55	0	0	17,9	36 unitarias; 19 de rebanada
ms-user-management	3007	20	0	0	13,4	8 unitarias; 12 de rebanada
ms-environment-monitoring	3003	92	0	0	52,5	28 unitarias; 55 Testcontainers; 9 de listeners
ms-user-experience	3006	10	0	0	20,8	10 Testcontainers
Total	—	**294**	**0**	**0**	**166,8**	**100 unitarias; 89 rebanada; 40 otros; 65 Testcontainers**

Tabla 4

Resumen por tipo de prueba

Tipo de prueba	Clases	Pruebas	% del total	Servicios
Unitaria pura	15	100	34,0 %	7
De rebanada (`@SpringBootTest` + H2)	9	89	30,3 %	5
Spring test con simulacros (servicios y filtros)	9	40	13,6 %	3
Testcontainers (PostgreSQL/RabbitMQ reales)	11	65	22,1 %	2
Total	**44**	**294**	**100 %**	**7**

La distribución se parece a la base de la pirámide de pruebas: un amplio piso de pruebas ligeras (unitarias + rebanadas = 64,3 %) y una punta pequeña de integración pesada. Lo que falta es la medición de cobertura que permita saber qué porcentaje del código ejecuta ese piso (hallazgo H1).

5. Inventario de las suites unitarias y trazabilidad con los requisitos

Las 15 clases unitarias puras se detallan a continuación con el comportamiento que verifican y su correspondencia con los requisitos y casos de uso del Entregable 02 (numeración del `03_Analisis_Software`).

Tabla 5

Suites unitarias puras, comportamiento verificado y requisitos cubiertos

Servicio	Clase	Pr.	Qué verifica	Requisitos (Entregable 02)
api-gateway	`JwtValidatorTest`	6	Validación del JWT en el borde: token firmado con otra clave, con `alg` manipulado o vacío rechazados; rutas públicas exentas	RF1.8, RF2.3 (RNF4)
ms-security	`JwtServiceTest`	7	Emisión RS256: claims correctos, `kid` coherente con el JWKS, expiración acorde al TTL, token alterado y token vencido rechazados, módulo y exponente RSA del JWKS	RF1.1, RF1.9
ms-security	`InstitutionServiceTest`	5	Crear institución: código en mayúsculas, duplicado rechazado, nombre en blanco rechazado, 404 para ID desconocido, actualización parcial	RF2.1
ms-classroom-management	`CampusServiceTest`	5	Sedes: código duplicado rechazado, código normalizado, borrado lógico, 404 para sede eliminada	RF2.2
ms-classroom-management	`EducationalEnvironmentCreatedEventTest`	5	Contrato del evento `educational.environment.created`: datos reales del agregado, `aggregateId`, routing key, sobre de eventos y `eventId` único	RF8.1, RF3.1

Servicio	Clase	Pr.	Qué verifica	Requisitos (Entregable 02)
ms-user-management	`UserProfileServiceTest`	8	Perfiles: `userId` duplicado rechazado, texto recortado, nombre completo obligatorio, actualización de campos provistos, borrado lógico	RF7.1, RF2.4
ms-sensor-management	`SensorServiceTest`	8	Sensores: serie única, serie recortada, borrado bloqueado con historial de instalación (409), modelo y estado desconocidos rechazados	RF8.2
ms-sensor-management	`SensorInstallationServiceTest`	9	Instalaciones: solapamiento rechazado, instalación abierta persistida, cierre con `removedAt`, rechazos de cierre inválido y publicación (o no) de los eventos instalado/ removido	RF8.2
ms-sensor-management	`SensorCatalogServiceTest`	9	Catálogo: modelos, estados y unidades; borrado bloqueado cuando el elemento está en uso	RF8.2
ms-sensor-management	`VariableServiceTest`	5	Variables: código duplicado, recorte de texto, borrado bloqueado si sensores la referencian, unidad de medición obligatoria	RF8.2, RF3.2
ms-sensor-management	`SensorInstallationEventTest`	5	Contrato de los eventos de instalación: carga útil, marcas de tiempo, routing key y sobre serializado	RF8.2, RF9.1
ms-environment-monitoring	`EnvironmentalAnalysisTest`	10	Máquina de estados del análisis ambiental: `PENDING` → `RUNNING` → `COMPLETED/FAILED`, ventanas no vacías, resultados adjuntos solo en ejecución	RF3.4
ms-environment-monitoring	`PeriodResolverTest`	12	Resolución de períodos día/semana/mes/año, año bisiesto, rollover de diciembre a enero, ventana válida, período nulo rechazado	RF3.4, INV-007

Servicio	Clase	Pr.	Qué verifica	Requisitos (Entregable 02)
ms-environment-monitoring	`EnvironmentAlertTest`	3	Alertas: nueva alerta activa, reconocimiento (`acknowledge`) la desactiva y no puede repetirse	RF6.2
ms-environment-monitoring	`EnvironmentalDataAnalyzedTest`	3	Evento de resultado de análisis: carga, normalización de resultados nulos y copia defensiva	RF3.4

Dos observaciones de trazabilidad: (a) las clases de contrato de eventos (`*EventTest`) protegen la frontera asíncrona entre servicios, que es donde el acoplamiento es más frágil; (b) los requisitos de seguridad (RF1.1, RF1.8, RF1.9) tienen cobertura unitaria en `ms-security` y en el borde (`JwtValidatorTest`), pero el bloqueo de cuenta y la rotación de refresh token se prueban en las rebanadas, no en las unitarias puras.

6. Lo que no es unitario (y por qué importa distinguirlo)

6.1 Pruebas de rebanada sobre H2 (89 pruebas)

Nueve clases `@SpringBootTest` con MockMvc arrancan el contexto de Spring completo del servicio sobre H2 en memoria (Liquibase deshabilitado, esquema creado por Hibernate). Cubren los controladores: `AuthControllerTest` (20), `CampusControllerTest` (15), `SensorControllerTest` (13), `UserProfileControllerTest` (12), `EducationalEnvironmentControllerTest` (8), `SensorFlowControllerTest` (6), `PasswordFlowTest` (5), `OAuth2EndpointTest` (4) y `GatewayIntegrationTest` (6). Son rápidas, pero dependen del contexto de la aplicación: no deben confundirse con unitarias.

6.2 Pruebas con Testcontainers (65 pruebas)

`ms-environment-monitoring` y `ms-user-experience` validan repositorios, listeners de RabbitMQ y controladores contra **PostgreSQL 15 real** (y RabbitMQ real en monitoreo) mediante Testcontainers. Incluyen `SchemaVerificationTest` (9), que verifica que el esquema generado coincide con el modelo esperado, y `ThresholdServiceTest` (7), que prueba la evaluación de umbrales contra datos reales. Son las más lentas (52,5 s y 20,8 s) y **exigen Docker**: sin él, esos dos servicios no pueden verificar su suite completa.

6.3 El smoke E2E: `infra/scripts/smoke.sh`

El script no es una prueba unitaria ni corre en CI. Es una verificación de humo **extremo a extremo** contra el stack levantado (`docker compose up -d`) que requiere `jq` y el SUPER_ADMIN sembrado por arranque. Secuencia: `GET /health` del gateway → login del SUPER_ADMIN → creación de una institución temporal → registro de un usuario con el código de esa institución → login → `logout` (revocación) → reintento con el token revocado, que **debe** responder `401 TOKEN_REVOKED`. Cubre el flujo crítico de identidad (CU-02, RF1.1, RF1.9) de punta a punta, algo que ninguna suite unitaria puede hacer.

7. Integración continua

La tubería activa vive en `infra/.github/workflows/build-services.yml`. Por cada repositorio `EduAirControl/ea-control-*` hace checkout, configura JDK 17 (Temurin) y ejecuta `./mvnw -B verify`; si el evento es un push a `main`, construye y publica la imagen en GHCR.

Tabla 6

Cobertura de la matriz de CI frente a los servicios existentes

Servicio	¿En la matriz de CI?	Corre con `clean`	Comentario
api-gateway	Sí	No	Riesgo H2
ms-security	Sí	No	Riesgo H2

Servicio	¿En la matriz de CI?	Corre con `clean`	Comentario
ms-classroom-management	Sí	No	Riesgo H2
ms-sensor-management	Sí	No	Riesgo H2
ms-user-management	Sí	No	Riesgo H2
ms-environment-monitoring	Sí	No	Requiere Docker para Testcontainers (lo ofrece el runner)
ms-user-experience	**No**	—	**Omitido de la matriz** (hallazgo H3)

El `smoke.sh` no aparece en ninguna tubería: se ejecuta manualmente (hallazgo H4).

8. Hallazgos y recomendaciones

Tabla 7

Hallazgos de la revisión del 11 de octubre de 2026

#	Hallazgo	Evidencia	Impacto	Recomendación
H1	No hay medición de cobertura de código	Cero configuraciones de JaCoCo en los siete `pom.xml`	No se puede verificar RNF6 (≥ 80 % de líneas, ≥ 90 % en lógica de negocio)	Añadir el plugin JaCoCo con umbral de fallo en el pipeline
H2	El CI ejecuta `verify` sin `clean`	Hoy, 3 servicios produjeron 64 errores espurios por clases obsoletas; con `clean` pasan	Falsos rojos o falsos verdes depending del runner	Cambiar a `./mvnw -B clean verify` en la matriz

#	Hallazgo	Evidencia	Impacto	Recomendación
H3	`ms-user-experience` no está en la matriz de CI	La matriz lista 6 servicios; el séptimo existe y tiene 10 pruebas	Sus pruebas no se ejecutan en cada push/PR	Añadir el servicio a la matriz
H4	El smoke E2E no está automatizado	`smoke.sh` solo se invoca manualmente (README de infra)	La verificación del flujo crítico depende de una persona	Ejecutarlo en el pipeline de staging o como job manual (`workflow_dispatch`)
H5	La copia de trabajo de `api-gateway` no compila	Edición sin commitear en `ratelimit/RedisStateStoreImpl.java` (sintaxis rota)	Impide verificar el servicio localmente	Terminar o revertir el cambio; la suite contra HEAD está en verde (27/27)
H6	`MICROSERVICIOS.md` está desactualizado	Declara «139 pruebas» y marca `ms-environment-monitoring` y `ms-user-experience` como pendientes	Información engañosa para quien llegue al proyecto	Actualizar el conteo a 294 y los estados de los servicios

9. Conclusiones

1. Las pruebas unitarias del backend están **en verde**: 100 pruebas unitarias puras en 15 clases, distribuidas en los siete servicios, con 0 errores y 0 fallos en la corrida del 11 de octubre de 2026.
2. El backend completo suma **294 pruebas automatizadas** (unitarias, de rebanada y de integración con contenedor), cifra que duplica la que declara la documentación interna y que debe actualizarse (hallazgo H6).
3. El `smoke.sh` **no es una prueba unitaria**: es la verificación E2E del flujo de identidad y debe tratarse como una capa distinta, complementaria y hasta hoy manual.

4. La principal carencia no es de cantidad de pruebas sino de **medición**: sin JaCoCo no hay porcentaje de cobertura y el requisito RNF6 no es verificable (hallazgo H1).
5. Dos ajustes de un renglón tienen retorno inmediato: `clean` en el pipeline (H2) y `ms-user-experience` en la matriz de CI (H3).

Referencias

Docker. (n.d.). *Docker documentation*. https://docs.docker.com

EduAirControl. (2026). *Especificación de requisitos de software de EduAirControl* (Entregable 02) [Documento interno del proyecto].

EduAirControl. (2026). *Informe de pruebas unitarias del backend* (Entregable 07) [Documento interno del proyecto].

JUnit. (n.d.). *JUnit 5 documentation*. https://junit.org/junit5/docs/current/user-guide/

Mockito. (n.d.). *Mockito*. https://mockito.org

Maven. (n.d.). *Surefire plugin*. https://maven.apache.org/surefire/

OWASP. (n.d.). *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

Sonar. (n.d.). *JaCoCo*. https://www.jacoco.org/jacoco/

Testcontainers. (n.d.). *Testcontainers*. https://testcontainers.com

Anexo A. Comandos de la corrida

```
# Verificar un servicio (H2, sin Docker)
cd ms-security && ./mvnw -B clean verify

# Verificar los dos servicios con Testcontainers (requiere Docker)
cd ms-environment-monitoring && ./mvnw -B clean verify
cd ms-user-experience && ./mvnw -B clean verify

# Ver api-gateway sin tocar la copia de trabajo
cd api-gateway && git worktree add --detach /tmp/gw-head develop
cd /tmp/gw-head && ./mvnw -B clean verify

# Smoke E2E (requiere el stack completo y jq)
cd infra && docker compose up -d && ./scripts/wait-for-services.sh
&& ./scripts/smoke.sh
```

Anexo B. Detalle por clase

Tabla 8

Las 44 clases de prueba con sus pruebas y tipo

Servicio	Clase	Pr.	Tipo
api-gateway	JwtValidatorTest	6	Unitaria pura
api-gateway	GatewayIntegrationTest	6	De rebanada
api-gateway	RateLimitFilterTest	6	Filtro con simulacros
api-gateway	PublicPathsTest	4	Filtro con simulacros
api-gateway	RemoteJwksProviderTest	3	Filtro con simulacros
api-gateway	CorrelationIdFilterTest	2	Filtro con simulacros
ms-security	AuthControllerTest	20	De rebanada
ms-security	AuthServiceTest	11	Servicio con simulacros
ms-security	JwtServiceTest	7	Unitaria pura
ms-security	PasswordFlowTest	5	De rebanada
ms-security	InstitutionServiceTest	5	Unitaria pura

Servicio	Clase	Pr.	Tipo
ms-security	OAuth2EndpointTest	4	De rebanada
ms-security	RedisTokenBlacklistTest	3	Servicio con simulacros
ms-security	SuperAdminBootstrapTest	2	Servicio con simulacros
ms-classroom-management	CampusControllerTest	15	De rebanada
ms-classroom-management	EducationalEnvironmentControllerTest	8	De rebanada
ms-classroom-management	CampusServiceTest	5	Unitaria pura
ms-classroom-management	EducationalEnvironmentCreatedEventTest	5	Unitaria pura
ms-sensor-management	SensorControllerTest	13	De rebanada
ms-sensor-management	SensorCatalogServiceTest	9	Unitaria pura
ms-sensor-management	SensorInstallationServiceTest	9	Unitaria pura
ms-sensor-management	SensorServiceTest	8	Unitaria pura
ms-sensor-management	SensorFlowControllerTest	6	De rebanada
ms-sensor-management	SensorInstallationEventTest	5	Unitaria pura
ms-sensor-management	VariableServiceTest	5	Unitaria pura
ms-user-management	UserProfileControllerTest	12	De rebanada
ms-user-management	UserProfileServiceTest	8	Unitaria pura
ms-environment-monitoring	AnalysisControllerTest	12	Testcontainers
ms-environment-monitoring	PeriodResolverTest	12	Unitaria pura
ms-environment-monitoring	EnvironmentalAnalysisTest	10	Unitaria pura
ms-environment-monitoring	HistoricalBackfillServiceTest	10	Testcontainers

Servicio	Clase	Pr.	Tipo
ms-environment-monitoring	SchemaVerificationTest	9	Testcontainers
ms-environment-monitoring	MonitoringControllerTest	8	Testcontainers
ms-environment-monitoring	ThresholdServiceTest	7	Testcontainers
ms-environment-monitoring	ClassroomEnvironmentListenerTest	5	Listener con simulacros
ms-environment-monitoring	SensorInstallationListenerTest	4	Listener con simulacros
ms-environment-monitoring	AlertServiceTest	4	Testcontainers
ms-environment-monitoring	MeasurementAlertingServiceTest	4	Testcontainers
ms-environment-monitoring	EnvironmentAlertTest	3	Unitaria pura
ms-environment-monitoring	EnvironmentalDataAnalyzedTest	3	Unitaria pura
ms-environment-monitoring	MsEnvironmentMonitoringApplicationTests	1	Testcontainers
ms-user-experience	UserExperienceControllerTest	7	Testcontainers
ms-user-experience	UxBackfillServiceTest	2	Testcontainers
ms-user-experience	MsUserExperienceApplicationTests	1	Testcontainers